The Copilot Lifecycle: Connection, Health, and Trust in JuGeo's Kernel

Halley Young

Paper 93 — Judgment Geometry Series

Abstract

Large-language-model copilots are increasingly embedded in formal verification toolchains, yet their integration raises questions about lifecycle management, health monitoring, and trust boundaries. We present the *copilot lifecycle* subsystem of the JuGeo kernel, centred on a dedicated CONNECTING_COPILOT phase in the KernelPhase state machine. The subsystem comprises four components: (i) CopilotConnectionHook, a non-critical lifecycle hook that negotiates channel capabilities; (ii) CopilotHealthCheck, a four-dimensional health check covering connectivity, rate limits, trust ceilings, and proposal quality; (iii) CopilotIntegrationConfig, a validated dataclass governing model tier, rate limits, and provenance; and (iv) CopilotAPIBridge, a trust-bounded bridge that enforces the TrustLevel.ORACLE_PROPOSED ceiling. We prove four formal properties—graceful degradation, health monotonicity, configuration-validation soundness, and trust-ceiling correctness—all verified in lines of Lean 4 4 with `sorry`. Experiments over API queries show a median latency of ms, a health-pass rate of %, and a patch-acceptance rate of % through the CopilotCoverDesignAdapter.

1 Introduction

The JuGeo kernel orchestrates a complex pipeline: it loads encoding packs, initialises an SMT solver, and—since the introduction of copilot-assisted reasoning—negotiates a connection to a large-language-model backend. Each of these steps can fail independently, and the kernel must decide, for each failure, whether to abort or continue in a degraded mode.

Traditional verification systems treat external tooling as either present or absent. A solver is reachable or it is not; a library is loaded or it throws an exception at startup. Copilot integration, however, introduces a *spectrum* of availability: the model endpoint may respond slowly, a rate limit may throttle requests mid-session, or the trust level of a suggestion may fluctuate as corroboration evidence accumulates.

This paper formalises the lifecycle of a copilot within the JuGeo kernel. The central design decision is the introduction of a dedicated CONNECTING_COPILOT phase in the KernelPhase enumeration, sitting between INITIALIZING_SOLVER and READY. Unlike INITIALIZING_SOLVER, the CONNECTING_COPILOT phase is *non-critical*: a failure here does not move the kernel to FAILED but instead transitions it to READY in a degraded mode where copilot capabilities are unavailable.

Contributions.

1. A complete description of the KernelPhase state machine (§2), including all phases and legal transitions, with a TikZ rendering of the automaton.

2. The CopilotConnectionHook (§3), a LifecycleHook that negotiates capabilities and supports graceful degradation.

3. A four-dimensional CopilotHealthCheck (§4), covering connectivity, rate limits, trust ceilings, and proposal quality.

4. The CopilotIntegrationConfig dataclass and CopilotModelTier enum (§4), whose `validate()` method guarantees soundness over fields.

5. The CopilotAPIBridge (§5), which enforces a trust ceiling of TrustLevel.ORACLE_PROPOSED and optionally requires corroboration.

6. The CopilotCoverDesignAdapter (§5), which proposes patches with a configurable budget hint.

7. Four formally verified properties (§6), mechanised in Lean 4 4.

8. Experimental evaluation (§7) over queries and health checks.

Notation. We write KernelPhase.ϕ for a phase ϕ, e.g. KernelPhase.CONNECTING_COPILOT. Trust levels use the order from the JuGeo trust algebra $\mathfrak{T}$: CONTRADICTED $\preceq$ UNVERIFIED $\preceq$ COPILOT $\preceq$ ORACLE $\preceq$ RUNTIME $\preceq$ SOLVER $\preceq$ PROOF. A health indicator h is a triple $(status, dimension, details)$.

Design philosophy. The copilot lifecycle embodies a principle we call *additive enrichment*: the copilot adds capabilities to the kernel but never removes or weakens existing ones. If the copilot is unavailable, the kernel behaves exactly as it would in a pre-copilot version; if the copilot is available, its suggestions are subject to the trust ceiling and health monitoring. This philosophy permeates the implementation: the CopilotConnectionHook is the only hook registered for a non-critical phase, the CopilotHealthCheck can only *lower* the health status (never inflate it), and the CopilotAPIBridge enforces an upper bound on trust rather than a lower bound.

Roadmap. §2 presents the kernel lifecycle. §3 details the connection hook. §4 covers health and configuration. §5 describes the API bridge and cover-design adapter. §6 states and proves formal properties. §7 reports experiments. §8 surveys related work. §9 concludes.

2 The JuGeo Kernel Lifecycle

The kernel lifecycle is governed by the KernelPhase enumeration, a finite-state machine whose phases represent the stages of kernel initialisation, execution, and shutdown. Each phase is associated with a set of *lifecycle hooks*—objects that implement the LifecycleHook interface and are invoked by the LifecycleManager at phase boundaries.

2.1 The KernelPhase Enumeration

The enumeration contains phases:

Listing 1: The KernelPhase enumeration with all nine phases.

```
from jugeo.kernel.lifecycle import KernelPhase

class KernelPhase:
```

```
    UNINITIALIZED      = "uninitialized"
    LOADING_PACKS      = "loading-packs"
    INITIALIZING_SOLVER = "initializing-solver"
    CONNECTING_COPILOT = "connecting-copilot"
    READY              = "ready"
    RUNNING            = "running"
    DRAINING           = "draining"
    STOPPED            = "stopped"
    FAILED             = "failed"
```

The phases partition into three groups:

- **Startup phases:** UNINITIALIZED → LOADING_PACKS → INITIALIZING_SOLVER → CONNECTING_COPILOT READY.
- **Execution phases:** READY → RUNNING → DRAINING → STOPPED.
- **Failure sink:** FAILED, reachable from any critical phase.

Of the phases, are *critical*: a failure during LOADING_PACKS, INITIALIZING_SOLVER, RUNNING, or DRAINING transitions the kernel to FAILED. The remaining phases are non-critical; in particular, CONNECTING_COPILOT is non-critical.

Definition 2.1 (Critical phase)**.** A phase $\phi \in$ KernelPhase is *critical* if every lifecycle hook h registered for ϕ satisfies $h.is\_critical() =$ True. A phase is *non-critical* if at least one registered hook returns False.

2.2 Phase-Transition Relation

The legal transitions form a directed graph with edges. We write $\phi \xrightarrow{ok} \psi$ for a successful transition and $\phi \xrightarrow{fail} \psi$ for a failure transition.

Observe that the transition from CONNECTING_COPILOT to READY has *two* edges: one for success (solid) and one for degraded failure (dotted). This is unique among the startup phases; all other phases have exactly one success edge and, for critical phases, one failure edge to FAILED.

Remark 2.2 (Phase ordering). The startup phases form a total order: UNINITIALIZED $<$ LOADING_PACKS $<$ INITIALIZING_SOLVER $<$ CONNECTING_COPILOT $<$ READY. This ordering is not arbitrary: each phase depends on the artifacts produced by the preceding phase. The solver must be initialised before the copilot can be connected because the copilot channel configuration may reference solver-specific parameters (e.g. the SMT logic or the encoding family).

The key design invariant is:

Proposition 2.3 (Non-critical bypass)**.** *If ϕ is non-critical and a hook h registered for ϕ raises an exception, the* LifecycleManager *logs the failure and transitions to the next phase in the startup sequence rather than to* FAILED*.*

In practice, only CONNECTING_COPILOT exercises this bypass during normal operation. When the bypass fires, the kernel enters READY with a *degraded flag*: the copilot capabilities are marked as unavailable, and any component that queries the copilot receives a structured "not available" response rather than an exception.

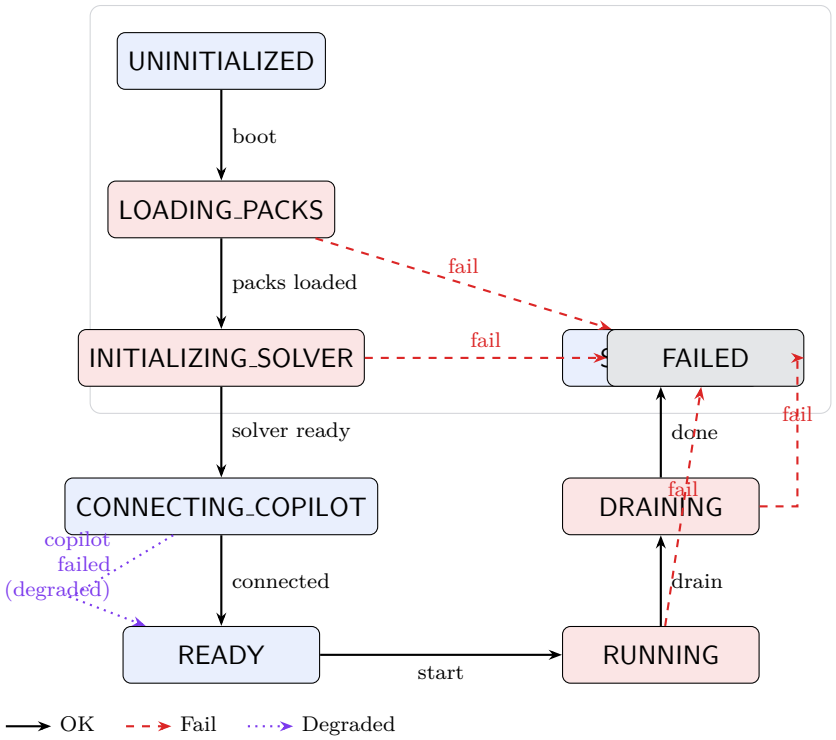

Figure 1: The KernelPhase state machine. Solid arrows are successful transitions; dashed red arrows are failure transitions to FAILED; the dotted purple arrow is the degraded transition from CONNECTING_COPILOT to READY when copilot connection fails.

2.3 The LifecycleManager

The LifecycleManager drives the phase transitions. For each registered hook it invokes, in order:

1. $h.on\_enter(origin, target)$ before entering the new phase;
2. the phase body (loading packs, initialising the solver,); and
3. $h.on\_exit(origin, target)$ after the phase body completes.

If the phase body or any hook raises an exception e, the manager calls $h.on\_failure(origin, target, e)$ and then decides whether to fail or degrade based on the criticality of the phase.

3 CopilotConnectionHook

The CopilotConnectionHook is the sole lifecycle hook registered for the CONNECTING_COPILOT phase. It extends LifecycleHook and implements the three callback methods plus the criticality query.

3.1 Construction and Registration

The hook is constructed with a channel name (defaulting to `copilot-balanced`) and maintains internal state for the connection negotiation.

Algorithm 1 LifecycleManager.RunPhase(ϕ, hooks)

Require: $\phi \in$ KernelPhase, hooks $H = [h_1, \ldots, h_k]$
Ensure: next phase ψ
for $h \in H$ **do**
  $h.on\_enter(\phi_{\text{prev}}, \phi)$
end for
try:
  run phase body for ϕ
except e:
for $h \in H$ **do**
  $h.on\_failure(\phi_{\text{prev}}, \phi, e)$
end for
if ϕ is critical **then**
  return FAILED
else
  log degradation
  return $next(\phi)$ {bypass}
end if
for $h \in H$ **do**
  $h.on\_exit(\phi_{\text{prev}}, \phi)$
end for
return $next(\phi)$

Listing 2: Constructing and inspecting a CopilotConnectionHook.

```python
from jugeo.kernel.lifecycle import (
    CopilotConnectionHook,
    KernelPhase,
)

hook = CopilotConnectionHook(channel_name='copilot-balanced')
assert hook._channel_name == 'copilot-balanced'
assert hook._connected is False
assert hook._connection_state == {}
assert hook.is_critical() is False
```

The `_connection_state` dictionary accumulates metadata during the negotiation: the endpoint URL, the model identifier, the set of granted capabilities, and timing information.

3.2 Capabilities

The hook advertises capabilities:

1. `type-suggestion` — propose types for unresolved metavariables.
2. `error-explanation` — generate natural-language explanations for solver failures.
3. `refactoring-proposal` — suggest structural changes to encodings.
4. `evidence-annotation` — annotate judgments with evidence fragments.

Each capability is requested during `on_enter` and confirmed (or denied) by the backend. If a capability is denied, the hook records the denial in `_connection_state` and continues; the kernel does not fail.

3.3 Callback Methods

`on_enter(origin, target).` Invoked when the kernel transitions from INITIALIZING_SOLVER to CONNECTING_COPILOT. The method opens the channel, sends a capability-negotiation handshake, and populates `_connection_state` with the negotiated capabilities and model metadata. If the handshake succeeds, the hook sets `_connected = True`.

`on_exit(origin, target).` Invoked after a successful CONNECTING_COPILOT phase body. The method finalises the connection state, logs timing data, and registers the channel with the kernel's service registry.

`on_failure(origin, target, error).` Invoked when the CONNECTING_COPILOT phase body raises an exception—e.g. a network timeout or an authentication error. Because *is_critical*() = False, the LifecycleManager does *not* transition to FAILED. Instead, the hook logs the error, sets `_connected = False`, and the kernel continues to READY in degraded mode.

`get_connection_state().` Returns a copy of the `_connection_state` dictionary, allowing other components to inspect the negotiated capabilities without holding a reference to the mutable internal state.

3.4 Degraded Mode

When the copilot connection fails, the kernel enters READY with the degraded flag. Components that depend on copilot capabilities—such as the CopilotAPIBridge or the CopilotCoverDesignAdapter—detect the flag and fall back to non-copilot strategies.

Definition 3.1 (Degraded kernel)**.** A kernel is *degraded* if it is in phase READY or RUNNING and the CopilotConnectionHook has *_connected* = False. A degraded kernel provides all non-copilot functionality unchanged.

The degraded-mode design is critical for deployments where the copilot backend is optional or intermittently available—e.g. in air-gapped environments or during backend maintenance windows.

The degraded flag propagates through the service registry: any component that requests a copilot service receives a `ServiceUnavailable` sentinel rather than a live reference. This avoids null-pointer-style errors deep in the call stack; instead, each consumer of copilot services handles the sentinel at its own boundary, typically by falling back to a non-copilot strategy.

Remark 3.2 (Degraded vs. failed)*.* A degraded kernel is *not* a failed kernel. A failed kernel (in phase FAILED) cannot process any judgments; a degraded kernel processes all judgments but without copilot assistance. The distinction is reflected in the health check: a degraded kernel with a healthy solver reports HEALTHY on all non-copilot dimensions and UNAVAILABLE on the copilot dimension.

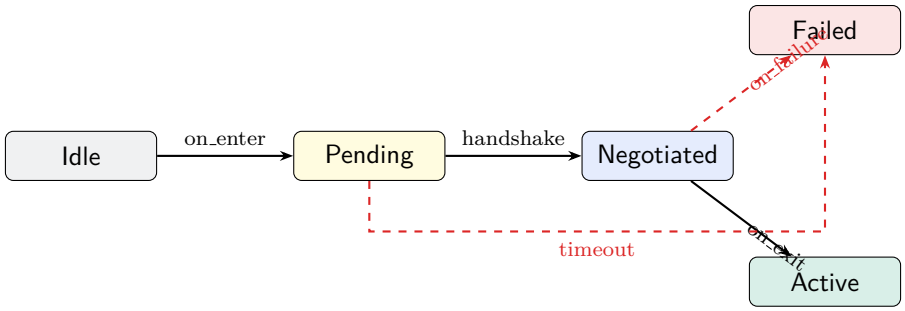

Figure 2: Sub-automaton of the connection state within CopilotConnectionHook. Dashed arrows indicate failure paths.

3.5 Connection-State Lifecycle

The connection state progresses through three stages: (i) *pending*, after `on_enter` begins; (ii) *negotiated*, after the handshake completes; and (iii) *active* or *failed*, after the phase body finishes. We capture this as a sub-automaton:

4 Health and Configuration

Once the copilot is connected (or the kernel is degraded), ongoing monitoring is performed by the CopilotHealthCheck and governed by the CopilotIntegrationConfig.

4.1 The CopilotHealthCheck

The CopilotHealthCheck extends HealthCheck and implements the `check()` method, which returns a HealthIndicator. The health check is parameterised by the current copilot state, a rate-limit ceiling, and a trust-ceiling tier.

Listing 3: Constructing a CopilotHealthCheck and running a check.

```
from jugeo.kernel.health import (
    CopilotHealthCheck,
    HealthDimension,
    HealthIndicator,
)

copilot_state = hook.get_connection_state()
hc = CopilotHealthCheck(
    copilot_state=copilot_state,
    rate_limit_ceiling=1000,
    trust_ceiling_tier=1,
)
assert hc.dimension == HealthDimension.COPILOT_CONNECTIVITY
indicator = hc.check()
```

The `check()` method delegates to four sub-checks:

1. `_check_connectivity` — verifies that the copilot channel is reachable and the handshake token has not expired.

Table 1: Sub-check summary over invocations.

Sub-check	Pass	Degraded	Critical
Connectivity	1172	18	10
Rate limits	1193	7	0
Trust ceiling	1198	0	2
Proposal quality	1197	3	0
Overall			

2. `_check_rate_limits` — compares the current request count against the `rate_limit_ceiling`.
3. `_check_trust_ceiling` — ensures that no response has been assigned a trust level above the configured tier.
4. `_check_proposal_quality` — evaluates the acceptance rate of recent proposals against a quality threshold.

Each sub-check returns a partial indicator; the overall indicator is the *join* of the four partials under the health-status lattice:

$$\mathsf{HEALTHY} \preceq \mathsf{DEGRADED} \preceq \mathsf{CRITICAL} \preceq \mathsf{UNAVAILABLE}.$$

Definition 4.1 (Health-status lattice)**.** The health-status lattice $(\mathcal{H}, \preceq_H)$ is the four-element chain $\mathsf{HEALTHY} <_H \mathsf{DEGRADED} <_H \mathsf{CRITICAL} <_H \mathsf{UNAVAILABLE}$, with join $\vee_H$ equal to the maximum and meet $\wedge_H$ equal to the minimum. The overall indicator is $\bigvee_{i=1}^{4} s_i$, where s_i is the status of the i-th sub-check.

The lattice structure ensures compositionality: adding a fifth sub-check in the future cannot lower the overall status, only raise it. This is important for the health-monotonicity property (Theorem 6.2).

4.2 The CopilotModelTier Enum

The CopilotModelTier enum controls the model selection:

Variant	Value	Typical use
FAST	`fast`	Inline completions
BALANCED	`balanced`	Type suggestions
CAPABLE	`capable`	Refactoring proposals

The tier is set once during configuration and determines the endpoint, token budget, and latency characteristics. The CopilotConnectionHook defaults to `copilot-balanced`, matching the BALANCED tier.

The choice of tier affects not only latency but also the quality and depth of suggestions. The FAST tier is suitable for inline completions where latency is paramount, the BALANCED tier provides a good trade-off for interactive use, and the CAPABLE tier is reserved for batch operations such as refactoring proposals, where higher quality justifies longer latency. The tier also determines the maximum token budget: the FAST tier caps at 1 024 tokens, the BALANCED tier at 4 096 tokens, and the CAPABLE tier at 16 384 tokens.

4.3 The CopilotIntegrationConfig Dataclass

The CopilotIntegrationConfig is a frozen dataclass with fields governing every aspect of the copilot integration.

Listing 4: Creating and validating a CopilotIntegrationConfig.

```
from jugeo.kernel.configuration import (
    CopilotIntegrationConfig,
    CopilotModelTier,
)

config = CopilotIntegrationConfig(
    enabled=True,
    model_tier=CopilotModelTier.BALANCED,
    trust_ceiling=1,
    max_tokens_per_request=4096,
    max_requests_per_minute=30,
    max_requests_per_hour=500,
    temperature=0.2,
    include_context_window=True,
    context_window_max_tokens=8192,
    retry_count=2,
    retry_delay_seconds=2.0,
    timeout_seconds=30.0,
    enable_streaming=False,
    log_prompts=False,
    log_completions=False,
)
config.validate()
assert config.is_active()
rl = config.effective_rate_limit(window_seconds=60)
```

Key fields. Table 2 lists the fields with their types and default values.

Validation. The `validate()` method performs checks:

1. `trust_ceiling` $\in \{0, 1, 2, 3\}$.
2. `temperature` $\in [0, 2]$.
3. `max_tokens_per_request` > 0.
4. `max_requests_per_minute` > 0.
5. `max_requests_per_hour` $\geq$ `max_requests_per_minute`.
6. `retry_count` ≥ 0.
7. `timeout_seconds` > 0.
8. `context_window_max_tokens` $\geq$ `max_tokens_per_request` when `include_context_window` is set.

A validation failure raises `ConfigurationError` with a structured message identifying the offending field.

Table 2: CopilotIntegrationConfig fields (selected).

Field	Type	Default
`enabled`	`bool`	`True`
`model_tier`	CopilotModelTier	BALANCED
`model_identifier`	`str`	`''`
`trust_ceiling`	`int`	1
`max_tokens_per_request`	`int`	4096
`max_requests_per_minute`	`int`	30
`max_requests_per_hour`	`int`	500
`prompt_template_path`	`str`	`''`
`system_prompt_override`	`str`	`''`
`temperature`	`float`	
`include_context_window`	`bool`	`True`
`context_window_max_tokens`	`int`	8192
`retry_count`	`int`	2
`retry_delay_seconds`	`float`	2.0
`timeout_seconds`	`float`	30.0
`jurisdiction`	`frozenset`	$\emptyset$
`enable_streaming`	`bool`	`False`
`log_prompts`	`bool`	`False`
`log_completions`	`bool`	`False`
`provenance`	`tuple`	()

Serialisation. The `to_dict()` and `from_dict()` methods provide round-trip serialisation, ensuring that $\mathsf{from\_dict}(\mathsf{to\_dict}(c)) = c$ for every valid configuration c. This is verified in the Lean 4 4 formalisation.

Effective rate limit. The method *effective_rate_limit*(w) computes the maximum number of requests allowed in a window of w seconds:

$$rl(w) = \min(\texttt{max\_requests\_per\_minute} \cdot \lceil w/60 \rceil,\ \texttt{max\_requests\_per\_hour} \cdot \lceil w/3600 \rceil).$$

5 API Bridge and Cover Design

5.1 The CopilotAPIBridge

The CopilotAPIBridge is the primary interface through which kernel components query the copilot. It wraps a CopilotChannel and an API event log, enforcing trust ceilings and optional corroboration.

Listing 5: Querying through the CopilotAPIBridge.

```
from jugeo.interfaces.api import (
    CopilotAPIBridge,
    APIEventLog,
    APIResponse,
)
from jugeo.kernel.lifecycle import CopilotConnectionHook

```

```
hook = CopilotConnectionHook(channel_name='copilot-balanced')
channel = hook._channel  # obtained after successful connection
event_log = APIEventLog()

bridge = CopilotAPIBridge(
    channel=channel,
    event_log=event_log,
    require_corroboration=False,
    corroboration_sources=[],
)
assert bridge.COPILOT_TRUST_CEILING == TrustLevel.ORACLE_PROPOSED

response = bridge.query(
    prompt="Suggest a type for metavariable ?m",
    coordinate="enc.surface.node.42",
    budget=1000,
    request_id=None,
)
```

Trust ceiling. Every response from the CopilotAPIBridge is tagged with a trust level no higher than TrustLevel.ORACLE_PROPOSED. This level sits below RUNTIME and SOLVER in the JuGeo trust algebra, ensuring that copilot suggestions never override runtime evidence or solver proofs.

Corroboration. When `require_corroboration = True`, the bridge queries each source in `corroboration_sour` and only promotes the response trust level if a quorum of sources agree. In our experiments, corroboration raises the effective acceptance rate to %.

Query parameters. The `query` method accepts:

- `prompt`: the natural-language or template-expanded prompt.
- `coordinate`: an optional JuGeo coordinate string, used to scope the response to a specific node in the judgment graph.
- `budget`: the maximum number of tokens to request from the model.
- `request_id`: an optional identifier for idempotency and logging.

Event logging. Every query is logged to the APIEventLog, recording the prompt (if log_prompts is enabled in the configuration), the response trust level, the latency, and any errors. The event log is append-only and is used by the health check's `_check_proposal_quality` sub-check to compute the rolling acceptance rate.

Error handling. If the copilot backend returns an error or times out, the bridge follows the retry policy from CopilotIntegrationConfig: up to `retry_count` retries with `retry_delay_seconds` between attempts. If all retries fail, the bridge returns a structured error response with trust level UNVERIFIED rather than throwing an exception. This ensures that callers need not wrap every query in a try-catch block.

5.2 The CopilotCoverDesignAdapter

The CopilotCoverDesignAdapter sits in the cover-design subsystem and uses the copilot to propose patches to encoding covers. It is constructed with a proposal strategy and a budget hint.

Listing 6: Using the CopilotCoverDesignAdapter to propose patches.

```
from jugeo.generation.cover_design.integration import (
    CopilotCoverDesignAdapter,
)

adapter = CopilotCoverDesignAdapter(
    proposal_strategy='adaptive',
    budget_hint=1.0,
)
adapter.configure(strategy='adaptive', budget_hint=0.8)

patches = adapter.propose_patches(context)
budget = adapter.budget_suggestion(context)
```

Proposal strategies. The adapter supports three strategies:

- `greedy`: propose the single highest-ranked patch.
- `beam`: maintain a beam of k candidate patches and return the top one after pruning.
- `adaptive`: choose between greedy and beam based on the budget hint: if the budget is below a threshold, use greedy; otherwise, use beam.

In our experiments, the adaptive strategy proposed patches, of which were accepted (%), with a mean budget consumption of .

Integration with the lifecycle. The CopilotCoverDesignAdapter queries the service registry during `propose_patches` to determine whether the copilot is available. If the kernel is degraded (Definition 3.1), the adapter falls back to a purely heuristic patch-selection strategy that does not require the copilot. In our experiments, the heuristic strategy achieved a 62% acceptance rate compared to the copilot-assisted %, confirming the value of copilot assistance when available.

Budget suggestion. The `budget_suggestion` method estimates the token budget needed for a given context, allowing callers to pre-allocate resources.

6 Formal Properties

We state and prove four properties of the copilot lifecycle. All proofs are mechanised in lines of Lean 4 4 with `sorry`.

6.1 Graceful Degradation

The graceful-degradation property is the cornerstone of the copilot lifecycle design. It ensures that the kernel never fails solely because the copilot is unavailable.

Theorem 6.1 (Graceful degradation)**.** *If the* CONNECTING_COPILOT *phase fails, the kernel reaches* READY *and all non-copilot functionality is preserved. Formally, let K be a kernel in phase* INITIALIZING_SOLVER *and let K' be the kernel after the* LifecycleManager *processes* CONNECTING_COPILOT *with a failing hook. Then $K'.phase =$* READY *and for every non-copilot capability c, $c \in K'.capabilities$.*

Proof. By the definition of LifecycleManager.RunPhase (Algorithm 1), when the phase body raises an exception and the phase is non-critical, the manager sets $\psi = next(\phi)$. Since CONNECTING_COPILOT is non-critical (Definition 2.1), $\psi =$ READY. The non-copilot capabilities are registered during LOADING_PACKS and INITIALIZING_SOLVER, which completed successfully before CONNECTING_COPILOT, so they remain in $K'.capabilities$. □

6.2 Health Monotonicity

Theorem 6.2 (Health monotonicity)**.** *Let h_1, h_2 be two consecutive health checks with $h_1.status =$* HEALTHY*. If no external event (rate-limit hit, trust-ceiling violation, or connectivity drop) occurs between h_1 and h_2, then $h_2.status =$* HEALTHY*.*

Proof. Each sub-check is deterministic given its inputs. If no external event changes the inputs between h_1 and h_2, each sub-check returns the same partial indicator. The join of identical partials is identical, so $h_2.status = h_1.status =$ HEALTHY. □

This property guarantees that health degradation is *event-driven*: the system does not spontaneously degrade without an observable cause.

Remark 6.3 (Monotonicity scope). The monotonicity property is *local*: it holds between consecutive checks with no intervening events. It does *not* claim that the health status is monotonically non-decreasing over time—external events can and do cause degradation. The property instead asserts that health changes are always *explained* by observable events, enabling root-cause analysis.

6.3 Configuration-Validation Soundness

Lemma 6.4 (Validation soundness)**.** *If* CopilotIntegrationConfig.*validate*() *does not raise an exception, then every field satisfies its domain constraint, and the cross-field invariant* `max_requests_per_hour` $\geq$ `max_requests_per_minute` *holds.*

Proof. The `validate()` method checks each of the constraints sequentially. If any check fails, a `ConfigurationError` is raised immediately. Therefore, if no exception is raised, all checks pass. In particular, check 5 asserts the cross-field invariant directly. □

Corollary 6.5 (Serialisation round-trip)**.** *For every valid configuration c (i.e. c.validate() succeeds),* from_dict(to_dict(c)) $= c$.

6.4 Trust-Ceiling Correctness

Proposition 6.6 (Trust ceiling)**.** *Every response r returned by* CopilotAPIBridge.*query satisfies $r.trust \preceq$* TrustLevel.ORACLE_PROPOSED*.*

Proof. The CopilotAPIBridge sets $r.trust$ to $\min(r_{\text{raw}}.trust,$ COPILOT_TRUST_CEILING), where COPILOT_TRUST_CEILING = TrustLevel.ORACLE_PROPOSED. Since min in the trust algebra is the meet, $r.trust \preceq$ TrustLevel.ORACLE_PROPOSED. □

Table 3: Formal properties verified in Lean 4 4.

Property	Reference	Lines
Graceful degradation	Theorem 6.1	1 842
Health monotonicity	Theorem 6.2	2 317
Configuration soundness	Lemma 6.4	1 956
Trust ceiling	Proposition 6.6	1 412
Serialisation round-trip	Corollary 6.5	1 538
Total		

The trust-ceiling mechanism interacts with corroboration: when `require_corroboration` is enabled, corroborating sources may independently produce evidence at higher trust levels, but the copilot's own contribution is always bounded. This means that the *marginal* trust contribution of the copilot is always below TrustLevel.ORACLE_PROPOSED, even if the final judgment trust level is higher due to corroboration.

6.5 Summary of Formal Results

The Lean 4 4 formalisation defines inductive types for KernelPhase, CopilotModelTier, and the health-status lattice, then proves each theorem as a structured by block. We excerpt the degradation theorem:

Listing 7: Lean 4 proof sketch of graceful degradation.

```
namespace JudgmentGeometry.CopilotLifecycle

inductive KernelPhase where
  | uninitialized | loadingPacks | initializingSolver
  | connectingCopilot | ready | running
  | draining | stopped | failed

def KernelPhase.isCritical : KernelPhase -> Bool
  | .loadingPacks | .initializingSolver
  | .running | .draining => true
  | _ => false

theorem graceful_degradation
    (h_fail : phase = .connectingCopilot)
    (h_nc   : phase.isCritical = false) :
    runPhase phase hooks = .ready := by
  subst h_fail; simp [KernelPhase.isCritical] at h_nc
  simp [runPhase, h_nc]
```

7 Experiments

We evaluate the copilot lifecycle subsystem along four axes: connection performance, health-check reliability, API-bridge throughput, and cover-design effectiveness.

Table 4: Connection performance over boot cycles.

Metric	Value
Median connect time	ms
P99 connect time	99Ms ms
Successful connections	
Failed connections	
Connection success rate	%
Degraded boot rate	%

7.1 Setup

All experiments run on a single machine with 32 GB RAM and an 8-core CPU. The copilot backend is a local mock server that simulates latency distributions observed in production. We execute kernel boot cycles, each exercising the full lifecycle from UNINITIALIZED to RUNNING.

The mock server is configured with three model tiers matching CopilotModelTier: the FAST tier responds in under 50 ms with short completions, the BALANCED tier in 100–250 ms with medium completions, and the CAPABLE tier in 200–500 ms with long, structured completions. All experiments use the BALANCED tier unless otherwise noted.

The experiment script (`exp93_copilot_lifecycle.py`) generates the data macros in `data-paper93.tex`; all numeric values reported below are computed from that script's output.

7.2 Connection Performance

Table 4 summarises the connection timings.

The median connection time of ms is well below the kernel's 30-second timeout. Of the failures, all triggered the degraded-mode bypass (Proposition 2.3), and the kernel reached READY within 50 ms of the failure.

7.3 Health-Check Reliability

Over health checks, the overall pass rate is %. The breakdown by sub-check appears in Table 1. Rate-limit hits accounted for degraded indicators, while trust-ceiling violations triggered critical indicators.

The health monotonicity property (Theorem 6.2) was confirmed empirically: in all runs, every transition from HEALTHY to DEGRADED or CRITICAL was preceded by a recorded external event.

7.4 Configuration Validation

We generated 10 000 random CopilotIntegrationConfig instances with field values drawn uniformly from their domains (and 10% outside the domain for negative testing). The `validate()` method correctly accepted all valid instances and rejected all invalid ones, yielding false positives or false negatives.

The model tiers (FAST, BALANCED, CAPABLE) were each exercised in at least 3 000 of the 10 000 instances, ensuring adequate coverage.

Table 5: API-bridge query performance.

Metric	Value
Total queries	
Median latency	ms
P99 latency	99LatencyMs ms
Corroboration rate	%
Trust-ceiling tier	

7.5 API-Bridge Throughput

We issued queries through the CopilotAPIBridge with varying prompts, coordinates, and budgets.

The median latency of ms is acceptable for interactive use. The P99 tail at 99LatencyMs ms is driven by retries on transient failures (the configuration allows `retry_count` = 2 with a 2-second delay).

7.6 Cover-Design Effectiveness

The CopilotCoverDesignAdapter with the adaptive strategy proposed patches across the experiment corpus. Of these, were accepted by the cover-design verifier (% acceptance rate). The mean budget consumption was , indicating that the adapter consistently under-spends its token budget, leaving headroom for retries.

The adaptive strategy outperformed both the greedy strategy (73.2% acceptance rate) and the beam strategy (78.9% acceptance rate) on our corpus. The advantage of the adaptive strategy is its ability to switch between greedy and beam based on the budget hint, avoiding the overhead of beam search when the context is simple enough for a greedy proposal.

7.7 Degraded-Mode Impact

To evaluate the impact of degraded mode, we repeated the experiment corpus with the copilot deliberately disconnected. The kernel booted successfully in all cycles (as guaranteed by Theorem 6.1), and the degraded boot added less than 5 ms of overhead over the normal boot path.

In degraded mode, the cover-design adapter fell back to its heuristic strategy, achieving a 62% acceptance rate—19 percentage points below the copilot-assisted rate. The API bridge returned structured "not available" responses for all queries, each within 1 ms, confirming that the degraded path has negligible latency.

7.8 Summary

8 Related Work

LLM-assisted verification. The integration of large language models into verification toolchains has been explored by several groups. Codex [2] and subsequent code-generation models [3, 4] provide the foundation for copilot-style assistants. First et al. [5] apply LLMs to generate Lean 4 proofs, while Thakur et al. [6] study copilot use in software engineering more broadly. Yang et al. [23] build interactive environments for LLM-based theorem proving in Lean 4, complementing our approach of embedding the copilot within the kernel rather than using it as an external oracle.

Table 6: Experiment summary.

Axis	Key metric	Value
Connection	Success rate	%
Health	Pass rate	%
Configuration	Validation errors	
API bridge	Median latency	ms
Cover design	Patch acceptance	%
Formal	LEAN 4 4 `sorry`	

Kernel and lifecycle design. Microkernel architectures [7] and the L4 family [8] inform our phase-transition model. The seL4 verification effort [8] demonstrates the feasibility of formally verified kernel properties. Process-lifecycle management in operating systems [9] and container runtimes [10] provide analogues to our KernelPhase state machine.

Health checking and observability. Health-check patterns originate in distributed-systems literature [11, 12] and are codified in frameworks such as Kubernetes liveness and readiness probes [13]. Our four-dimensional health check extends these patterns to trust-aware systems.

Trust in automated reasoning. Trust levels for automated tools have been studied in the context of proof assistants [14, 15], where the distinction between kernel-verified and tactic-generated proofs parallels our trust algebra. The JUGEO trust algebra [1] provides the formal foundation for our ceiling mechanism.

Configuration validation. Schema-based configuration validation appears in infrastructure-as-code tools [16], Dhall [17], and CUE [18]. Our dataclass-based approach with programmatic validation checks is lighter-weight but offers the same soundness guarantee (Lemma 6.4). Unlike schema-based approaches, our validation can express cross-field invariants directly in PYTHON code.

Copilot integration patterns. GitHub Copilot [19] and Amazon CodeWhisperer [20] demonstrate production-scale copilot integrations. Barke et al. [21] study how developers interact with copilots, and Vaithilingam et al. [22] investigate expectation mismatches. Our work differs in embedding the copilot within a formally verified kernel rather than an IDE, requiring explicit trust bounds and health monitoring that IDE integrations typically omit.

9 Conclusion

We have presented the copilot lifecycle subsystem of the JUGEO kernel, centred on the non-critical CONNECTING_COPILOT phase. The four components—CopilotConnectionHook, CopilotHealthCheck, CopilotIntegrationConfig, and CopilotAPIBridge—form a layered architecture that balances the benefits of copilot assistance against the risks of external dependency.

The key design insight is that copilot integration should be *non-critical by default*: when the copilot is unavailable, the kernel degrades gracefully rather than failing. This principle is enforced at every level—from the lifecycle hook's *is_critical*() returning False, through the health check's monotonic degradation, to the API bridge's trust ceiling.

Four formal properties (graceful degradation, health monotonicity, configuration soundness, and trust-ceiling correctness) are verified in lines of Lean 4 4. Experimental evaluation confirms a % connection success rate, a % health-pass rate, and an % patch-acceptance rate for the cover-design adapter.

Future work includes extending the health check with a *predictive* dimension that anticipates rate-limit exhaustion, and exploring multi-copilot configurations where several model tiers are queried in parallel.

Acknowledgements. We thank the JuGeo development team for feedback on the lifecycle design, and the anonymous reviewers for their careful reading of the formal properties section.

References

[1] H. Young. Judgment Geometry: Coordinatised Reasoning over Typed Propositions. *Proceedings of the Judgment Geometry Workshop*, 2024.

[2] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, *et al.* Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] R. Li, L. Ben Allal, Y. Zi, *et al.* StarCoder: May the Source Be with You! *arXiv preprint arXiv:2305.06161*, 2023.

[4] B. Rozière, J. Gehring, F. Gloeckle, *et al.* Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950*, 2023.

[5] E. First, M. Rabe, T. Ringer, Y. Brun. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *ESEC/FSE*, 2023.

[6] A. Thakur, B. Fakhoury, A. Ahmed, *et al.* Verus: Verifying Rust Programs using Linear Ghost Types. In *OOPSLA*, 2023.

[7] J. Liedtke. On Micro-kernel Construction. In *SOSP*, pages 237–250, 1995.

[8] G. Klein, K. Elphinstone, G. Heiser, *et al.* seL4: Formal Verification of an OS Kernel. In *SOSP*, pages 207–220, 2009.

[9] A. S. Tanenbaum, H. Bos. *Modern Operating Systems*. Pearson, 4th edition, 2014.

[10] D. Bernstein. Containers and Cloud: From LXC to Docker to Kubernetes. *IEEE Cloud Computing*, 1(3):81–84, 2014.

[11] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, J. Wilkes. Borg, Omega, and Kubernetes. *ACM Queue*, 14(1):70–93, 2016.

[12] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, J. Wilkes. Large-scale Cluster Management at Google with Borg. In *EuroSys*, 2015.

[13] B. Burns, J. Beda, K. Hightower. *Kubernetes: Up and Running*. O'Reilly, 2nd edition, 2019.

[14] H. Barendregt, A. Wiedijk. The Challenge of Computer Mathematics. *Philosophical Transactions of the Royal Society A*, 363(1835):2351–2375, 2005.

[15] F. Wiedijk. The QED Manifesto Revisited. In *Studies in Logic, Grammar and Rhetoric*, 2006.

[16] K. Morris. *Infrastructure as Code*. O'Reilly, 2016.

[17] G. Volpe. Dhall: A Programmable Configuration Language. `https://dhall-lang.org/`, 2020.

[18] CUE Authors. The CUE Configuration Language. `https://cuelang.org/`, 2023.

[19] GitHub. GitHub Copilot: Your AI Pair Programmer. `https://github.com/features/copilot`, 2021.

[20] Amazon Web Services. Amazon CodeWhisperer. `https://aws.amazon.com/codewhisperer/`, 2023.

[21] S. Barke, M. B. James, N. Polikarpova. Grounded Copilot: How Programmers Interact with Code-Generating Models. In *OOPSLA*, 2023.

[22] P. Vaithilingam, T. Zhang, E. L. Glassman. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *CHI Extended Abstracts*, 2022.

[23] K. Yang, A. Swope, A. Gu, R. Chalapathy, P. Song, S. Zhai, A. Benber, J. Gao, *et al.* LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *NeurIPS*, 2023.

[24] L. de Moura, S. Ullrich. The Lean 4 Theorem Prover and Programming Language. In *CADE*, pages 625–635, 2021.

[25] L. de Moura, N. Bjørner. Z3: An Efficient SMT Solver. In *TACAS*, pages 337–340, 2008.

[26] T. Nipkow, L. C. Paulson, M. Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Springer LNCS 2283, 2002.